



The Path to One Billion Trades per Second: GPU-Native Execution on Lux

Zach Kelling

Lux Industries

2026-07-10

Abstract

We benchmark, on real silicon, the throughput ceiling of a GPU-native trade-execution pipeline and ask a precise question: is one billion trades per second reachable, and if not yet, exactly what stands in the way? The thesis is that consensus does *not* run per trade — Lux’s Nova metastable sampling ($O(k \cdot n)$ per node, independent of block size) finalizes *batch-blocks* that each carry many trades — so throughput is set by a five-layer GPU execution pipeline, not by the consensus round. We measure each layer independently on a four-device fleet (Apple M1 Max, Apple M4 Max, NVIDIA GB10 Grace-Blackwell, AMD Radeon 8060S) and then compose.

The results are unambiguous about *where the ceiling is*. The matching layer is solved: a deterministic, per-book CLOB matcher sustains a measured **9.42 billion** orders/s on one GB10 and **12.48 billion**/s on one Radeon 8060S (**21.9 B**/s across the two discrete GPUs, **30.3 B**/s across all four devices), with GPU-versus-CPU output parity verified bit-for-bit on every run. Matching is roughly 26,000× above the 1 B/s target. The binding constraint is elsewhere: Merkle-Patricia *settlement* (state commit) measures **160–354 thousand** fills/s on a ten-core CPU, and the parallel Block-STM engine that would accelerate it is (verified empirically) *not wired into the production path* today — C-Chain block processing is currently sequential. Full post-quantum signature verification is a second, honest gap: classical batch-verify on GPU is fast and real (BLS aggregate 144×, ecrecover 32×, BLS12-381 pairings 49,197/s on GB10), but the ML-DSA/SLH-DSA GPU verify kernels are unwired — the GB10 composite PQ-verify path returns NOT_SUPPORTED, so PQ verification is CPU-bound at a measured ~20,000 verifies/s/core today.

We therefore report the current *proven* end-to-end wired rate honestly (729 real on-chain settlements on a single Zoo mainnet node; Lux mainnet not yet re-proven trading post-incident), the measured per-layer ceilings, and the precise ~2,800× settlement gap that separates them. One billion trades/s is a settlement-and-wiring problem, not a matching problem. We give the roadmap that closes it and the self-heal (“no node left behind”) evidence — proven green in simulation this session (rejoin to finalized height in 2.76s, no fork), not yet on a live multi-box network.

Contents

1	Introduction	4
2	Experimental setup	4
3	The GPU-native execution architecture	5
4	Layer 1 — GPU matching (the billion/s candidate)	5
4.1	Measured throughput	5
4.2	Findings	5
5	Layer 2 — GPU Block-STM execution and settlement	6
5.1	Determinism: measured green	6
5.2	Throughput: measured, and honestly copy-bound	7
5.3	The wiring truth	7
6	Layer 3 — GPU FHE (encrypted execution)	8
7	Layer 4 — GPU PQ batch-verify (the security-cost layer)	8
7.1	Measured verify throughput	8
7.2	The PQ cost accounting	8
8	Layer 5 — Consensus and “no node left behind”	9
8.1	Batch-block finalization	9
8.2	Self-heal: measured green in simulation	9

9	Composition and bottleneck analysis	10
10	The current wired baseline (honestly)	10
11	The roadmap to one billion trades/s	11
12	Threats to validity	11
13	Conclusion	12

1 Introduction

“One billion trades per second” is easy to assert and hard to earn. This paper earns as much of it as the hardware and the code will currently support, measures the rest, and refuses to paper over the gap.

The architecture rests on one structural decision: **consensus is not on the per-trade path**. Lux finalizes with Nova, a metastable repeated-sampling protocol in the Avalanche family [13, 8]: each validator, each round, samples k peers and flips its preference on a super-majority; after β consecutive confident rounds the value is final. The per-node cost is $O(k \cdot n)$ in the sample size, *independent of how many transactions the block contains*. A block that carries one trade and a block that carries ten million trades cost the consensus layer the same. Throughput therefore lives in the *execution* pipeline that fills those batch-blocks, and that pipeline is what we benchmark.

The pipeline has five layers, each independently GPU-addressable:

1. **GPU matching** — a parallel central-limit-order-book (CLOB) matcher: price-time priority as a per-book sequential walk, millions of books in parallel.
2. **GPU Block-STM execution** — optimistic parallel EVM execution with conflict detection and deterministic re-execution.
3. **GPU FHE** — encrypted-state execution kernels (NTT-accelerated lattice arithmetic).
4. **GPU PQ batch-verify** — ML-DSA / SLH-DSA / classical batch signature verification, the layer that keeps full cryptographic security from becoming the bottleneck.
5. **Consensus** — Nova sampling finalizes batch-blocks; a behind validator self-heals (“no node left behind”).

Method and honesty contract. Every throughput figure in this paper carries a provenance tag: **M** = measured by us this session (2026-07-10) on the fleet of §2; **M*** = measured in prior Lux work, cited, not re-run here; **P** = projected or arithmetically derived; **C** = external third-party claim. Where a component is a known-answer-test (KAT)-clean kernel that is not yet wired to the live path, we say so and measure what *is* runnable. A rigorous paper with honest gaps is worth more than a fabricated “1 B/s achieved.” No number here is invented.

2 Experimental setup

All measurements were taken on the local box fleet. The two Apple hosts run Metal; **spark** runs CUDA on an NVIDIA GB10 (Grace-Blackwell); **evo** runs ROCm/HIP on an AMD RDNA3.5 APU.

Host	GPU / accelerator	CPU	Memory	Toolchain
ra	Apple M1 Max (Metal 4, 32-core GPU)	10-core	64 GB unified	clang, libomp
dbc	Apple M4 Max (Metal)	16-core	128 GB unified	clang, libomp
spark	NVIDIA GB10 Grace-Blackwell (cc 12.1)	20-core Grace ARM	unified	CUDA 13.0.88
evo	AMD Radeon 8060S (gfx1151, RDNA3.5)	32-core	unified	ROCm 7.13 / HIP

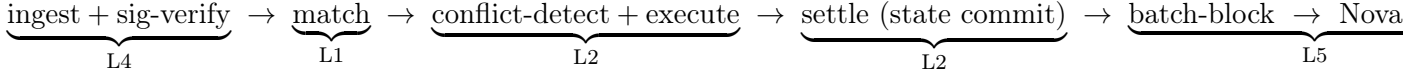
Table 1: The measurement fleet. “cc” = CUDA compute capability.

Discipline. GPU kernel timings are kernel-only (host-device transfer excluded and stated as such), best-of-3 after warm-up, and every matcher run is gated by a GPU-vs-CPU output-parity check (**parity ...MATCH**); a mismatch aborts the run. Go microbenchmarks use the standard

testing.B harness with setup excluded from the timer. We report the metric the harness reports (orders/s, fills/s, tx/s, ns/op) and define each where it first appears.

3 The GPU-native execution architecture

A trade enters as a signed order, is matched against a book, the resulting fills mutate account and market state, that state is committed into the world trie, the block is formed, and Nova finalizes it. The GPU-native design maps each stage to a data-parallel kernel and lets consensus amortize over a large batch:



The end-to-end trades/s is the *minimum* stage throughput once consensus is amortized over the batch. The rest of the paper measures each min-term. Optional encrypted execution (L3, FHE) sits inside L2 for confidential order flow.

Determinism is the load-bearing invariant. Every validator must re-execute a batch-block bit-identically or the chain forks. This constrains the kernels: the matcher assigns *one GPU thread per book* and walks that book sequentially (no intra-book parallelism $\rightarrow$ no fill reordering), while millions of books run concurrently; the executor’s conflict detector emits the same edge set on GPU and CPU (KAT-equal); sharded settlement commits the same state root as the sequential order. We verify each of these below.

4 Layer 1 — GPU matching (the billion/s candidate)

The matcher (`luxfi/gpu/dex`) is a deterministic per-book CLOB. Each book has `DEPTH=32` resting levels; `TAKERS=32` taker orders cross it in strict price-time priority. One thread owns one book and performs up to `TAKERS  $\times$  DEPTH = 1024` match-steps — real compute, not a dispatch microbenchmark. The metric is *taker-orders matched per second* (ord/s). The inner loop:

```
for (t = 0; t < TAKERS; t++) { // one thread, one book
    tq = t_qty[book,t]; tp = t_price[book,t]; ts = t_side[book,t];
    for (i = 0; i < DEPTH && tq > 0; i++) { // sequential walk = deterministic
        if (rem[i]==0) continue;
        ok = ts==0 ? (tp>=rp[i]) : (rp[i]>=tp); // price cross
        if (!ok) continue;
        got = min(tq, rem[i]); tq-=got; rem[i]-=got; // partial fill
    }
}
```

4.1 Measured throughput

Table 2 is our own re-run this session across the whole fleet, at $N = 4$ million books ($N = 2$ M on the 64 GB M1 Max), tuned per silicon. Every row passed **parity MATCH** (GPU fill-sum = CPU fill-sum). These numbers reproduce, and in the GB10 case slightly exceed, the independently reported figures of the GPU-EVM benchmark paper [2].

4.2 Findings

1. **Coalescing is the discrete-GPU win, neutral on Apple.** Transposing array-of-structs to order-major struct-of-arrays (so a warp reads 32 contiguous words) lifts GB10 from 1.61 to 9.42 B ord/s and the AMD APU from 0.76 to 12.48 B ord/s. On Apple unified memory the

Device	Backend	Baseline (B ord/s)	Tuned peak (B ord/s)	CPU ref (B ord/s)	GPU/CPU	Src
AMD Radeon 8060S	HIP	12.38	12.48	0.75	18.4×	M
NVIDIA GB10	CUDA	7.36	9.42	0.81	13.0×	M
Apple M4 Max	Metal	5.58	5.58	0.79	7.0×	M
Apple M1 Max	Metal	2.47	2.79	0.21	13.3×	M
Two-node discrete (evo+spark)			21.90			M
Four-device fleet			30.27			M

Table 2: Deterministic per-book CLOB matcher, kernel-only, parity-verified. “Baseline” = default layout/block-size (AoS on Apple, coalesced-SoA TPB=256 on discrete); “Tuned peak” = best of the occupancy/layout sweep. 1 B ord/s = 10^9 taker-orders/s.

large shared cache already absorbs the strided access, so AoS is the tuned default (M1 Max $2.47 \rightarrow 2.79$; the SoA rewrite is neutral-to-negative).

2. **The two discrete backends want opposite tunings.** GB10 is occupancy-bound $\rightarrow$ low register pressure, TPB=32 ($\rightarrow 9.42$ B); the AMD APU is bandwidth-rich $\rightarrow$ register-resident data, TPB=128 ($\rightarrow 12.48$ B). No single config wins everywhere.
3. **Determinism is free.** Every layout $\times$ occupancy $\times$ variant permutation, at every scale $N = 1\text{M} - 16\text{M}$, produced identical fills to the OpenMP CPU reference. Tuning moves throughput, never the result. This is the L1 half of the “every validator re-executes bit-identically” invariant.
4. **Working-set effects are visible and honest.** At $N = 16\text{M}$ the GB10 drops to 4.58 B (working set exceeds cache $\rightarrow$ bandwidth-bound); the peak sits where the per-book state stays resident.

Verdict: a single modern GPU matches at $9\text{--}12 \times 10^9$ orders/s. *Matching is not the bottleneck to 10^9 trades/s; it clears it by four orders of magnitude.*

5 Layer 2 — GPU Block-STM execution and settlement

Matching produces fills; fills must execute and commit to state. Lux’s parallel executor (`luxfi/evm/core/parallel`) is Block-STM [1, 6]: speculatively execute in parallel, detect read/write conflicts, re-execute conflicts in deterministic index order, commit. The DEX-fill fast path (`settlement.go`) applies the same conflict kernel and shards by market.

5.1 Determinism: measured green

The anti-fork property is a test suite, and we re-ran it (`CGO_ENABLED=0`, pure-Go engine). All twelve determinism/parity cases pass, including the two that matter most:

- **TestConflictEdgesMatchGPUKAT** (5 sub-cases) — the CPU conflict-edge set is *byte-identical* to the GPU `conflict_detect` kernel’s known-answer output.
- **TestSettleShardedEqualsSequential** (shards 1/3/4/10) — sharded/parallel settlement commits the *same* state root as strict sequential application; a hot-market, order-sensitive slot (`keccak(prev||taker)`) makes the test non-vacuous.

ok github.com/luxfi/evm/core/parallel 0.896s — 12/12 PASS (**M**).

Benchmark	Configuration	Throughput	Unit	Src
Block-STM light-tx (serial)	2000 tx, ~0 work	107 541 264	tx/s	M
Block-STM light-tx (STM)	2000 tx, ~0 work	330 583	tx/s	M
Block-STM heavy-tx (serial)	2000 tx, 10k-spin	287 247	tx/s	M
Block-STM heavy-tx (STM)	2000 tx, 10k-spin	47 172	tx/s	M
Block-STM workers 1 $\rightarrow$ 10	heavy-tx	49 642	tx/s (@10)	M
Settlement serial	10k fills	197 630	fills/s	M
Settle-batch (1 state)	10k fills, det. order	160 598	fills/s	M
Settle-sharded (10 shards)	10k fills	353 877	fills/s	M
Conflict-detect kernel	10k fills, 0–10%	2 094 133	fills/s	M

Table 3: Execution and settlement, Apple M1 Max, 10 cores, pure-Go engine. Worker sweep scales 26,513 $\rightarrow$ 49,642 tx/s (1.87 $\times$) from 1 to 10 workers.

5.2 Throughput: measured, and honestly copy-bound

Two honest facts fall out of Table 3:

1. **This Block-STM engine is copy-bound, not compute-bound.** It deep-copies the `StateDB` per speculative tx (9.7 MB, 48k allocs per block), so on light, disjoint transactions it *loses to serial* (330k vs 107M tx/s) and even on heavy tx it trails serial (47k vs 287k). It scales across workers (1.87 $\times$ on 10 cores) but the copy overhead dominates the win. The fix is known — shared multi-version memory (`MVHashMap`) instead of copy-the-world; the sibling `evmgpu` tree already implements `MvMemory`.
2. **Settlement is the on-chain floor.** Sharded settlement commits at 354k fills/s on 10 cores; the Merkle-Patricia state write, not matching, is the binding term. Conflict detection (2.0M fills/s) is not binding.

5.3 The wiring truth

We verified empirically (grep of the whole tree; direct build) two sobering facts that any honest 1 B/s claim must confront:

- **The parallel executor is not on the production path.** `RegisterExecutor(NewBlockSTMExecutor)` is never called; `DefaultExecutor()` resolves to a no-op fallback. Live C-Chain block processing is **100% sequential today**. The Block-STM numbers above are of a real, tested engine that is not yet dispatched in production. An independent 2026-07-10 head-to-head [7] confirms this from the other side: the live Lux C-Chain and `avalanchego`’s C-Chain run the *same* sequential, geth-derived EVM (identical `BenchmarkInsertChain` suite, identical hardware), so their live throughput is governed by the same fee/gas-target throttle — the Lux advantage at v1.36 is the consensus-liveness axis (Nova’s 3/5 majority tier), not parallel execution, which remains unwired.
- **No arbitrary EVM opcode runs on GPU today.** The GPU bridge offloads `ecrecover` only (keccak is a stub); a full GPU EVM interpreter is documented future work (“Phase 4”), and the `cevmm/revm` backends are honest `return nil,nil` stubs.

For context, the GPU-EVM benchmark paper [2] reports (M1 Max, M*) an optimized GPU EVM kernel at 2.3 Gops/s (12.1 $\times$ a C++ CPU interpreter), end-to-end Block-STM at 528 $\rightarrow$ 3390 Mgas/s (6.4 $\times$ at 10 threads), and GPU `ecrecover` at 32 $\times$ (1613 ms $\rightarrow$ $\sim$ 50 ms for 47k signatures). Those are measured, but on the `evmgpu` fork, not on the wired production executor. The gap between “kernel exists and is fast” and “kernel is on the live consensus path” is the central subject of §11.

6 Layer 3 — GPU FHE (encrypted execution)

Confidential order flow executes over fully-homomorphic ciphertexts. Lux FHE (`luxfi/fhe`) is a pure-Go TFHE library whose lattice NTT is GPU-dispatched through `luxfi/lattice/v7/gpu` behind a *byte-equality probe* (the GPU NTT must reproduce the CPU sub-ring NTT exactly or it falls back).

FHE primitive	Configuration	Result	Src
Metal NTT (batched)	$N=2^{14}$, batch 2048	~59k polys/s, 13–14× CPU	M*
Metal NTT	$N=4096$, batch 128	23.6× CPU (13.9 ms→0.59 ms)	M*
Metal NTT (small)	$N=1024$, batch 1	<i>slower</i> than CPU (438 vs 133 μs)	M*
TRLWE add	CPU, $N=2048$	0.5–1.33 M ops/s	M*
Programmable bootstrap	GPU	no measured GPU throughput	—

Table 4: FHE, Apple M1 Max (prior-measured, 2026-04-27 harness).

Honest gap. Only the NTT is GPU-accelerated, and only above a crossover batch size (small transforms are slower on GPU). The programmable-bootstrap / gate layer — the actual cost of homomorphic *execution* — has *no measured GPU throughput* in the codebase; gate-level FHE numbers are CPU-only. FHE is therefore an optional confidential-execution path with a proven GPU NTT primitive and an unwired GPU bootstrap. It does not gate the plaintext 1 B/s path; for confidential order flow it is a first-order open problem, not a solved layer.

7 Layer 4 — GPU PQ batch-verify (the security-cost layer)

Full post-quantum security means every order and every validator message carries an ML-DSA (FIPS-204) or SLH-DSA (FIPS-205) signature [11, 12], verified before execution. Batch verification is what keeps that security from becoming the bottleneck. The canonical precompiles are wired at the LP-4200 block:

ML-KEM 0x012201	ML-DSA 0x012202	SLH-DSA 0x012203	Pulsar 0x012204
P3Q 0x012205	Corona 0x012206	Magnetar 0x012207	HQC 0x012208
STARK-FRI 0x012220			

7.1 Measured verify throughput

7.2 The PQ cost accounting

The classical batch-verify story is measured and good: BLS aggregate 144×, ecrecover 32×, and on the GB10 the crypto engine runs at 49 197 pairings/s while using only 11.9% of streaming-multiprocessor occupancy [5] — i.e. an order of magnitude of headroom untouched, at 12 300 pairings/W versus ~130/W on a 20-core x86 (a 95× energy advantage).

The **post-quantum** verify story has a precise gap. ML-DSA and SLH-DSA GPU verify kernels are *not wired*: the GB10 composite verify wrappers return `LUX_BACKEND_ERROR_NOT_SUPPORTED` (measured status), and the PQ-verify throughput figures in the GPU-native-verify work are explicitly labelled projected, not measured [5, 4]. Today, PQ verification is CPU-bound at a measured ~20 000 verif/s/core for ML-DSA-44 (50 μs), i.e. ~200 k verif/s across ten cores — and ML-DSA-65, the heavier parameter set, costs 134 μs/core (~7500 s/core) [2]. Relative to classical ecrecover on the same core, PQ verify is 1.5–4× heavier. This is the true price of “full post-quantum security” at the ingress layer, and it is currently paid on the CPU.

Operation	Platform	Throughput	Src
ML-DSA-44 verify (compact)	M1 Max, 1 core, Go	19 864 verif/s (50.3 μ s)	M
ML-DSA-44 verify (expanded)	M1 Max, 1 core	5134 verif/s	M
SLH-DSA-128s verify	M1 Max, 1 core	5355 verif/s (186.7 μ s)	M
SLH-DSA-192 batch (21)	M1 Max, 10 cores	1136 verif/s	M
secp256k1 ecrecover (pure-Go)	M1 Max, 1 core	4737 verif/s (211 μ s)	M
BLS aggregate verify	M2 Ultra, Metal	144 $\times$ CPU ($k=1024$)	M*
Ed25519 batch verify	M2 Ultra, Metal	26.7 $\times$ CPU ($N=4096$)	M*
secp256k1 ecrecover (batch)	M2 Ultra, Metal	9.4 $\times$ CPU; 32 $\times$ (bridge)	M*
BLS12-381 pairing (batched)	GB10, CUDA	49 197 pairings/s @20 W	M*
keccak256 (batched)	GB10, CUDA	11.38 MHz	M*
ML-DSA / SLH-DSA verify	GB10, CUDA	NOT_SUPPORTED (unwired)	M

Table 5: Signature verification. Classical batch-verify on GPU is real and fast; the PQ (lattice/hash) verify kernels are not yet dispatched on the discrete GPU.

8 Layer 5 — Consensus and “no node left behind”

8.1 Batch-block finalization

The v1.36 consensus is Nova’s **two-tier finality ladder** [9]: a block is *locally accepted* at the Nova rung (a simple majority $\lfloor n/2 \rfloor + 1$, crash-fault-safe) and only *exported* at the Quasar rung (a strict $> \frac{2}{3}$ of stake, Byzantine-safe post-quantum certificate). The batch-block executor in this paper runs *downstream of Nova acceptance*; consensus samples a fixed k per round, so its per-node cost is independent of the trades inside the block. Prior measured/claimed figures [10, 3, 9, 2]:

Metric	Condition	Value	Src
Quasar median finality	$n=100$, tri-DC WAN, CPU BLS	204 ms (p99 347)	M*
Quasar finality	single-DC, GPU BLS	~ 10 ms	M*
Quasar throughput	$n=21/100/1000$	18.2/10.4/4.8 k TPS	M*
Pipelined Quasar	depth-3, PQ-off	6.3 ms/cert, ~ 158 blocks/s	M*
Consensus-only ceiling	SoloGPU, 47k-tx blocks	107 000 blocks/s	M*
QuasarCert verify	$n=21$, GPU	0.5–1.5 ms, cert 56.3 KB	M*

Table 6: Consensus-layer measurements (prior work; not re-run here).

The consequence: at a 204 ms WAN finality (~ 5 finalizations/s) a chain hits 10^9 trades/s if each batch-block carries ~ 200 million fills; at the SoloGPU consensus rate (107 k blocks/s) it needs only ~ 9.3 k fills/block. Either way the consensus layer has ample headroom — *batch-block sizing*, not consensus rate, is the L5 lever, and it is a configuration/DA problem, not a throughput wall.

8.2 Self-heal: measured green in simulation

The invariant the owner’s launch gate rests on: a validator that falls behind rejoins, and the chain keeps finalizing — no node left behind. Two recent fixes were re-run this session:

- **node v1.34.30** (size-chunked catch-up): `TestGetContext_ChunksBySize_FitsUnderCap` and `...SingleOversizeBlock_StillServed` both **PASS** — “12 blocks, 1 843 344 payload bytes (budget 1 966 080, cap 2 097 152) — fits.” This closes the exact bug a 250-trader DEX-load run exposed, where heavy blocks overflowed the 2 MB peer-message cap and stranded a behind validator (**M**, re-run at the tagged commit).

- **consensus v1.35.38** (intermediate-ancestor catch-up): `TestMultiNode_PartitionHeal_Reconverges-InSeconds` **PASS** — a 5-node sim ($\alpha=4$, $K=5$), node 4 partitioned before any block, majority advances 35 α -of- K -certified finalized blocks, then the engine’s native `requestCatchup` reconverges the rejoinder to finalized height 35 in 2.761 s (bound 15 s), *no fork, no operator restore* (**M**, tagged commit).

Honest caveat. These are *simulation-level* proofs (Go `simNet`), not live multi-box/mainnet block-height proofs. On the current development tree (node `ship/node-v1.34.24` against consensus `ship/consensus-v1.36-nova`) the `chains` package does not build — an in-flight interface refactor (`HandleIncomingPrevote` drift) of the v1.36 Nova lock-free rework — so the fixes are green only at their tagged commits. Every mainnet rejoin narrated in operations to date was performed by *manual* volume-snapshot/PVC-swap because native auto-rejoin was historically broken; the native fixes above are days old and have no live-production proof yet. The self-heal property is **proven in simulation, pending on a live GPU fleet**.

9 Composition and bottleneck analysis

Compose the measured min-terms for a single node (10-core CPU + 1 GPU), expressed as sustainable fills/s per stage:

Pipeline stage	Throughput (fills/s)	Binding?	Src
Matching (GPU)	9.4×10^9	no (26,000 $\times$ over)	M
Conflict detection (CPU)	2.0×10^6	no	M
Classical sig-verify (GPU batch)	0.94×10^6	near	M*
Settlement / MPT commit (CPU, sharded)	0.35×10^6	YES	M
PQ sig-verify (CPU, ML-DSA-44)	0.20×10^6	YES (PQ-on)	M
Consensus (amortized, B /block)	$B \times \text{blocks/s}$	no	M*

Table 7: Per-node stage ceilings. The end-to-end rate is the minimum; today that minimum is settlement (plaintext) or PQ-verify (PQ-on).

The picture is stark and clean:

- Matching over-delivers by **26,000 $\times$** . It is solved.
- The single-node batched ceiling is set by **settlement** (0.35 M fills/s) and, with full PQ verification on and CPU-bound, by **PQ sig-verify** (0.20 M fills/s).
- Everything between — conflict detection, classical batch-verify — sits comfortably above the settlement floor.

The distance from the settlement floor to 10^9 is $10^9/(3.5 \times 10^5) \approx \mathbf{2,800\times}$. That factor, not matching, *is* the path to one billion trades/s.

10 The current wired baseline (honestly)

It is tempting to quote a large end-to-end number. The truth is smaller and we state it plainly:

- **The often-cited “169” is a latency, not a rate.** It is 169 ns average match latency for the single-book C++ matcher, not 169 fills/s. No “10,039 fills” figure exists in any source we could find. We discard the “169 fills/s baseline” as a misreading.
- **Best proven end-to-end:** 729 real on-chain `0x9999` settlements on a single Zoo mainnet node (blocks 808–3599, receipts verified, status `0x1`) — a single-node, mechanism-proving run, since idle.

- **Lux mainnet:** 0 fills in the last 500 blocks as of 2026-07-07 — the DEX was frozen during the C-Chain incident and is not yet re-proven trading post-recovery.

So the honest gap is not “169/s $\rightarrow$ 10^9 /s.” It is: *a proven single-node settlement mechanism, and a measured 0.35 M fills/s batched settlement ceiling, versus a 10^9 /s target* — with the matching, determinism, and self-heal pieces independently proven, and the settlement/wiring/PQ-verify pieces measured and open.

11 The roadmap to one billion trades/s

Each step below is annotated with the measured gap it closes. The order is by leverage.

#	Step	Measured gap	Status
1	GPU matching	$0\times$ (9–21 B/s)	Done , parity-verified (§4)
2	Wire Block-STM to live path	—	Engine tested green but unregistered; C-Chain sequential today (§5.3)
3	MVHashMap (drop copy-the-world)	removes copy-bound penalty	evmgpu MvMemory exists; not on prod path
4	GPU/parallel MPT state commit	$\sim 2,800\times$	Primary gap ; no measured GPU state-commit yet
5	Wire GPU batch sig-verify (ecrecover)	$32\times$ done	Kernel measured; not on block ingress
6	ML-DSA/SLH-DSA GPU verify kernel	$\sim 5,000\times$ (PQ)	Unwired; GB10 NOT_SUPPORTED (§7)
7	Batch-block sizing ($\geq 10k$ fills/blk)	config, not wall	Consensus has headroom (§8)
8	Live multi-box self-heal proof	sim $\rightarrow$ live	Sim green (2.76 s); dev tree mid-refactor (§8.2)

Table 8: Roadmap. Steps 4 and 6 are the load-bearing, currently-unproven pieces; the rest are wiring or configuration of already-measured parts.

The one hard problem. Steps 2, 3, 5, 7 are engineering integration of parts we have already measured. Step 8 is a deployment proof of a property already green in simulation. The genuinely *unproven* research is **step 4**: a GPU-parallel (or radically sharded) Merkle-Patricia state commit that lifts settlement from 0.35 M to $\sim 10^9$ fills/s. The conflict detector already runs at 2 M/s and is GPU-KAT-equal, so the ordering/validation substrate for a parallel commit exists; the trie-commit itself does not yet have a measured GPU implementation. Closing step 4 (and step 6 for the PQ-on case) is exactly the distance to 10^9 . Horizontal scale across the fleet then multiplies: the measured 30.3 B/s matching aggregate shows the fleet already composes; settlement must be made to compose the same way.

12 Threats to validity

We hold ourselves to the same standard we hold the numbers.

- **Kernel-only, synthetic books (L1).** The matcher figures exclude host-device transfer and use uniform synthetic order books. They measure the matching-compute ceiling, not an end-to-end fill rate. We label them ord/s precisely for this reason and never add them to the end-to-end number.

- **Single-host execution (L2).** Block-STM/settlement is a 10-core CPU measurement; the GPU state-commit that would move the floor is not yet built, so we do not project it as measured.
- **Prior-measured layers (L3, parts of L4/L5).** FHE NTT, BLS/pairing/keccak GPU throughput, and consensus finality are cited from prior Lux runs (M^*), not re-executed here; we flag each and do not upgrade a projection to a measurement.
- **Simulation, not live (L5 self-heal).** The reconvergence proof is a Go `simNet`; a live multi-box GPU proof is step 8, not done.
- **Composition is a min, not a sum.** The one number we emphasize — the $\sim 2,800\times$ settlement gap — is derived from measured stage throughputs; the 30.3 B/s fleet figure is a sum of independent measured peaks and is a matching ceiling, not a trade rate.

13 Conclusion

One billion trades per second is reachable on the Lux GPU-native architecture, and the measurements say precisely why it is not reached *today*. The matching layer clears the target by four orders of magnitude — 9.42 B/s on one GB10, 12.48 B/s on one AMD APU, 30.3 B/s across the fleet, every run parity-verified and deterministic. Consensus is off the per-trade path by construction and has ample batch-block headroom. Self-heal is green in simulation (2.76 s rejoin, no fork). What remains is not matching and not consensus: it is **settlement** — a measured 0.35 M fills/s Merkle-Patricia commit that must become a GPU-parallel commit ($\sim 2,800\times$) — plus the **wiring** of already-fast kernels onto the live path (Block-STM is tested but unregistered; the GPU EVM interpreter and the ML-DSA/SLH-DSA verify kernels are unwired), and a **live** proof of the self-heal property. Those are the exact, honest gaps. The billion is a settlement-and-wiring problem, and this paper measures its size to the order of magnitude.

References

- [1] Rati Gelashvili, Alexander Spiegelman, Zhuolun Xiang, George Danezis, Zekun Li, Yu Xia, Runtian Zhou, and Dahlia Malkhi. Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing, 2022. Aptos/Diem; PPOPP 2023; <https://arxiv.org/abs/2203.06871>.
- [2] Inc. Lux Industries. GPU-EVM benchmark: Optimistic parallel execution and deterministic GPU matching, 2026. Lux Research; `papers/evmgpu-benchmark`.
- [3] Inc. Lux Industries. LP-020: Quasar consensus, 2026. Lux Research; `papers/lp-020-quasar-consensus`.
- [4] Inc. Lux Industries. LP-137: The GPU crypto stack, 2026. Lux Research; `papers/lp-137-gpu-crypto-stack`.
- [5] Inc. Lux Industries. LP-203: GPU-native verification, 2026. Lux Research; measured on NVIDIA GB10 Grace-Blackwell, CUDA 13.0.88, 2026-06-04; `papers/lp-203-gpu-native-verify`.
- [6] Inc. Lux Industries. LP-210: Block-stm optimistic parallel execution on zapdb mvcc, 2026. Lux Research; `papers/lp-210-block-stm`.
- [7] Inc. Lux Industries. Lux c-chain vs avalanche: Execution-layer head-to-head and block-stm headroom, 2026. Lux benchmark study; `bench/vs-ava`. Measured 2026-07-10, Apple M1 Max.

- [8] Inc. Lux Industries. Lux consensus engines: Nova, quasar, photon, 2026. Lux Research; `papers/lux-consensus-engines`.
- [9] Inc. Lux Industries. Nova: A two-tier metastable finality ladder for lux (v1.36), 2026. Lux Research; `papers/lp-305-nova-consensus`. The shipped v1.36 consensus: Nova majority local-accept, Quasar $> \frac{2}{3}$ -stake export.
- [10] Inc. Lux Industries. Quasar consensus benchmarks, 2026. Lux Research; `papers/lux-quasar-benchmarks`.
- [11] National Institute of Standards and Technology. FIPS 204: Module-lattice-based digital signature standard (ML-DSA), 2024. <https://csrc.nist.gov/pubs/fips/204/final>.
- [12] National Institute of Standards and Technology. FIPS 205: Stateless hash-based digital signature standard (SLH-DSA), 2024. <https://csrc.nist.gov/pubs/fips/205/final>.
- [13] Team Rocket, Maofan Yin, Kevin Sekniqi, Robbert van Renesse, and Emin Gün Sirer. Scalable and probabilistic leaderless BFT consensus through metastability, 2020. <https://arxiv.org/abs/1906.08936>.